

Algoritmi esercizi OS 2

Deadlock

L'algoritmo risolutivo fondamentale per gli esercizi sul deadlock è l'**Algoritmo del Banchiere** (o di verifica dello stato safe), strutturato in passaggi sequenziali per determinare se esiste una sequenza di esecuzione sicura.

Passo 1: Calcolo della matrice Need

- Calcola la matrice delle necessità sottraendo, per ogni processo P_i , le risorse già allocate da quelle massime richiedibili: $Need[i] = Max[i] - Alloc[i]$.

Passo 2: Ricerca di un processo idoneo

- Esegui un ciclo completo sui processi per cercare un P_i la cui necessità sia minore o uguale alle risorse attualmente disponibili: $Need[i] \leq Disponibili$.
- Se nessun processo soddisfa questa condizione e ci sono ancora processi da verificare, il ciclo si interrompe: il sistema è in uno stato **unsafe** (rischio deadlock).

Passo 3: Simulazione del rilascio delle risorse

- Quando trovi un processo idoneo, simula la sua esecuzione e il conseguente rilascio delle risorse aggiungendo le sue risorse allocate a quelle disponibili: $Disponibili = Disponibili + Alloc[i]$.
- Segna il processo come completato e passa al processo successivo.

Passo 4: Iterazione e conclusione

- Ripeti i Passi 2 e 3 finché non rimangono più processi da verificare.
- Se tutti i processi riescono a terminare, l'ordine in cui sono stati soddisfatti costituisce la **sequenza safe**, garantendo che il sistema al tempo T sia in uno stato sicuro.

Gestione di una nuova richiesta

Se l'esercizio chiede di valutare una richiesta esplicita di risorse da parte di un processo P_i , prima di applicare l'algoritmo principale devi eseguire questi controlli:

1. **Verifica liceità:** Controlla che $Richiesta \leq Need[i]$. In caso contrario, la richiesta non è lecita.
2. **Verifica disponibilità:** Controlla che $Richiesta \leq Disponibili$. In caso negativo, non ci sono risorse sufficienti e il processo deve attendere.
3. **Aggiornamento fittizio:** Se i controlli sono superati, simula l'assegnazione aggiornando i parametri:
 - $Disponibili = Disponibili - Richiesta$
 - $Alloc[i] = Alloc[i] + Richiesta$
 - $Need[i] = Need[i] - Richiesta$
4. Esegui i Passi 1-4 sui nuovi parametri: se lo stato finale risulta safe, la richiesta può essere soddisfatta; altrimenti si annulla l'operazione (rollback) allo stato iniziale.

Gestione della Memoria e Paginazione

1. Calcolo del Tempo Effettivo di Accesso (EAT - Effective Access Time)

Questo algoritmo serve per calcolare il tempo medio che la CPU impiega per recuperare un dato, considerando le cache (TLB) e i ritardi dei page fault.

- **Passo 1:** Identifica dal testo i parametri base: numero di livelli della page table (N), tempo di accesso della memoria fisica (T_{mem}) e tempo di accesso alla TLB (T_{tlb}).
- **Passo 2:** Estrai le probabilità di successo e fallimento della TLB: il testo fornisce l' Hit_ratio (percentuale di successo); calcola il $Miss_ratio$ sottraendolo dal 100% (es. $Miss_ratio = 1 - Hit_ratio$).
- **Passo 3:** Calcola l'EAT parziale (senza considerare i page fault) con questa formula esatta:
$$EAT = Hit_ratio \cdot (T_{mem} + T_{tlb}) + Miss_ratio \cdot ((N + 1) \cdot T_{mem} + T_{tlb}).$$
- **Passo 4:** Se l'esercizio introduce i page fault, identifica la percentuale in cui avvengono ($Page_fault_ratio$) e il tempo necessario per portare la pagina dal disco alla memoria (T_{medio}). Assicurati di convertire tutte le unità di misura (es. millisecondi in nanosecondi) affinché siano uniformi.
- **Passo 5:** Calcola il valore finale applicando la formula dell'EAT completo:
$$EAT_{completo} = (1 - Page_fault_ratio) \cdot EAT + Page_fault_ratio \cdot T_{medio}.$$

2. Traduzione degli Indirizzi (da Logico a Fisico)

Questo algoritmo serve per mappare un indirizzo generato dalla CPU in un indirizzo reale nella RAM.

- **Passo 1 (Ricerca del Numero di pagina):** Dividi l'indirizzo logico fornito per la dimensione in byte di una singola pagina e prendi solo la parte intera del quoziente. Formula:
$$Numero\ di\ pagina = \lfloor \frac{Indirizzo\ logico}{Dimensione\ pagina} \rfloor.$$
 In alternativa, se ti viene fornita una tabella con gli intervalli precalcolati, individua semplicemente in quale intervallo è contenuto l'indirizzo logico.
- **Passo 2 (Calcolo dell'Offset):** Calcola la posizione esatta del dato all'interno della pagina sottraendo dall'indirizzo logico l'inizio della pagina stessa. Formula: $Offset = Indirizzo\ logico - (Dimensione\ pagina \cdot Numero\ di\ pagina).$
- **Passo 3 (Associazione al Frame):** Consulta la tabella di paginazione (fornita dall'esercizio) per trovare quale $Numero\ di\ frame$ (pagina fisica) corrisponde al $Numero\ di\ pagina$ logica appena trovato.
- **Passo 4 (Calcolo finale):** Componi l'indirizzo fisico moltiplicando il frame per la grandezza della pagina e aggiungendo l'offset. Formula: $Indirizzo\ fisico = (Numero\ di\ frame \cdot Dimensione\ pagina) + Offset.$

3. Dimensionamento della Memoria Fisica (Numero di Pagine)

Questo procedimento si applica quando viene chiesta la capienza della memoria fisica in termini di posizioni di pagina.

- **Passo 1:** Converti la dimensione della pagina in bit di offset (n) calcolando il logaritmo in base 2 della sua dimensione in byte (es. pagina da 16 KB = 16384 B; $\log_2(16384) = 14$ bit di offset).
- **Passo 2:** Sottrai i bit dell'offset dal numero totale di bit dell'indirizzamento fisico indicati nel testo (es. 32 bit). Formula: $Bit\ di\ pagina\ fisica = Bit\ indirizzo\ fisico - n.$

- **Passo 3:** Calcola il numero massimo di posizioni di pagina elevando 2 alla potenza dei bit rimanenti calcolati al passo 2. Formula: $Totale\ pagine\ fisiche = 2^{Bit\ di\ pagina\ fisica}$.

4. Dimensionamento delle Tabelle delle Pagine (Voci per Tabella multi-livello)

Questo algoritmo è fondamentale per gli esercizi che richiedono di trovare la capienza di una singola tabella (di primo, secondo o ennesimo livello) in sistemi a più livelli.

- **Passo 1:** Ricava i bit di offset (n) come mostrato nell'algoritmo precedente e sottraili al numero totale di bit dell'indirizzamento *logico* (o virtuale) della CPU.
- **Passo 2:** Dividi i bit rimanenti in parti uguali, in base al numero di livelli della paginazione indicato dal testo (es. per un sistema a due livelli, dividi per 2). Questo ti darà il numero di bit che gestiscono un singolo livello di tabella.
- **Passo 3:** Calcola quante voci (entry) contiene una singola tabella elevando 2 alla potenza dei bit calcolati al passo 2. Formula: $Voci\ per\ tabella = 2^{Bit\ per\ singolo\ livello}$.

Sostituzione delle Pagine (Page Replacement)

Ecco gli algoritmi risolutivi dettagliati per calcolare il numero di page fault e gestire l'allocazione nei frame in base alle diverse politiche di rimpiazzamento.

Algoritmo Generale di Setup (Valido per tutti i metodi)

- **Passo 1:** Identifica nel testo il numero di frame (posizioni fisiche) inizialmente vuoti a disposizione in memoria (es. 3 posizioni).
- **Passo 2:** Analizza la sequenza di pagine richieste leggendole strettamente da sinistra verso destra, processandone una alla volta.
- **Passo 3:** Per ogni pagina letta, verifica se è già presente nei frame. Se è presente si verifica un *Hit* (non segnare nulla o segna successo); se non è presente si verifica un *Page Fault* (segna un errore, ad esempio con una "X").

Algoritmo FIFO (First In, First Out)

- **Logica base:** Si sostituisce sempre l'elemento presente da più tempo all'interno della memoria.
- **Passo 1 (Riempimento iniziale):** In caso di page fault, se ci sono frame vuoti, inserisci la pagina e memorizza il suo ordine di arrivo.
- **Passo 2 (Scelta della vittima):** Quando avviene un page fault e i frame sono tutti pieni, individua la pagina entrata per prima cronologicamente.
- **Passo 3 (Sostituzione):** Sovrascrivi la pagina più "vecchia" con quella appena richiesta.
- **Passo 4 (Aggiornamento stato):** La pagina appena inserita diventa la più "giovane", spostando l'indicatore della più vecchia sulla pagina che è entrata nel sistema subito dopo quella appena eliminata.

Algoritmo LRU (Least Recently Used)

- **Logica base:** Si sostituisce l'elemento che è stato utilizzato meno di recente nel passato.
- **Passo 1 (Tracciamento accessi):** Mantieni le pagine allocate nei frame costantemente ordinate per ultimo accesso. La pagina appena letta o inserita va considerata come l'accesso più recente.

- **Passo 2 (Aggiornamento su Hit):** Se la pagina richiesta è già in memoria (Hit), non c'è page fault, ma devi obbligatoriamente aggiornare il suo stato segnandola come la più usata di recente.
- **Passo 3 (Scelta della vittima):** In caso di page fault con frame pieni, guarda all'indietro nello storico degli accessi (o controlla il tuo ordine aggiornato) e individua la pagina con l'accesso meno recente.
- **Passo 4 (Sostituzione):** Sostituisci questa pagina con quella nuova, segna la "X" per il page fault e metti la nuova pagina in cima all'ordine di utilizzo.

Algoritmo Ottimo / MIN

- **Logica base:** Si sostituisce l'elemento che non verrà richiesto nel breve periodo, guardando al futuro della sequenza.
- **Passo 1 (Sguardo in avanti):** In caso di page fault con frame pieni, per ogni pagina attualmente presente in memoria, guarda in avanti nella parte restante della sequenza di richieste.
- **Passo 2 (Ricerca occorrenze):** Conta dopo quanti passi ogni pagina in memoria verrà richiesta di nuovo.
- **Passo 3 (Scelta della vittima):** Scegli da rimpiazzare la pagina allocata che verrà acceduta di nuovo più lontano nel tempo. Se una delle pagine in memoria non compare più in tutta la sequenza futura, scegliila immediatamente come vittima.
- **Passo 4 (Sostituzione):** Inserisci la nuova pagina al suo posto e segna il page fault con una "X".

File System e Inode

Ecco gli algoritmi risolutivi dettagliati per affrontare gli esercizi sui File System, in particolare il calcolo della dimensione massima di un file e il conteggio dei blocchi acceduti.

1. Calcolo della Dimensione Massima di un file

Questo algoritmo serve a calcolare la capienza massima (in byte) che un singolo file può raggiungere, data l'architettura dell'inode e la dimensione dei blocchi.

- **Passo 1 (Dimensione dell'indirizzo):** Identifica dal testo i bit per indirizzo e convertili in byte dividendo per 8 (es. indirizzi a 16 bit implicano $16 \div 8 = 2$ Byte per indirizzo).
- **Passo 2 (Indirizzi per blocco):** Calcola quanti puntatori (indirizzi) possono essere contenuti in un singolo blocco. Formula: $\text{Indirizzi per blocco} = \frac{\text{Dimensione blocco in Byte}}{\text{Byte per indirizzo}}$. Chiameremo questo valore N (es. $2048 B \div 2 B = 1024$ indirizzi).
- **Passo 3 (Calcolo blocchi indirizzabili):** Calcola il numero totale di blocchi dati che l'inode può puntare sommando le capacità dei blocchi diretti, indiretti singoli e indiretti doppi:
 - $\text{Blocchi diretti} = \text{Numero di indirizzi diretti}$.
 - $\text{Blocchi indiretti} = \text{Numero di indirizzi indiretti singoli} \cdot N$.
 - $\text{Blocchi indiretti doppi} = \text{Numero di indirizzi indiretti doppi} \cdot N^2$.
- **Passo 4 (Somma e risultato):** Somma tutti i blocchi calcolati al passo 3 per ottenere il *Totale blocchi indirizzabili*. Moltiplica questo totale per la dimensione in byte di un singolo blocco. Formula: $\text{Dimensione massima} = \text{Totale blocchi indirizzabili} \cdot \text{Dimensione blocco}$.

2. Conteggio dei Blocchi Unici Acceduti per leggere un file

Questo algoritmo si usa quando l'esercizio ti fornisce una dimensione specifica di un file (es. 3 GB) e ti chiede quanti blocchi fisici (sia dati che strutturali/indice) il sistema deve leggere dal disco.

- **Passo 1 (Blocchi dati totali):** Converti la dimensione del file in byte (es. $3\text{ GB} = 3.221.225.472\text{ B}$) e dividila per la dimensione del singolo blocco. Arrotonda per eccesso se c'è del resto. Formula: $\text{Blocchi dati} = \lceil \frac{\text{Dimensione file in Byte}}{\text{Dimensione blocco}} \rceil$.
- **Passo 2 (Sottrazione blocchi base):** Sottrai dai *Blocchi dati* i blocchi accessibili direttamente e quelli accessibili tramite tutti i puntatori indiretti singoli dell'inode (cioè $\text{Numero indiretti singoli} \cdot N$). Formula: $\text{Blocchi rimanenti} = \text{Blocchi dati} - \text{Indirizzi diretti} - (\text{Indirizzi indiretti singoli} \cdot N)$.
- **Passo 3 (Uso degli indiretti doppi):** Dividi i *Blocchi rimanenti* per la capacità di un singolo puntatore indiretto doppio (N^2) per scoprire quanti puntatori doppi ti servono. Formula: $\text{Indiretti doppi usati} = \lceil \frac{\text{Blocchi rimanenti}}{N^2} \rceil$. Se il risultato ha decimali, significa che ti servirà un intero puntatore doppio e un pezzo del successivo.
- **Passo 4 (Scomposizione puntatore parziale):** Sottrai dai *Blocchi rimanenti* i blocchi dati interamente gestiti dai puntatori doppi "pieni". Prendi il nuovo resto dei blocchi dati e dividilo per N , arrotondando per eccesso. Formula: $\lceil \frac{\text{Nuovo resto blocchi}}{N} \rceil$. Questo ti dà il numero di blocchi indiretti singoli (strutturali) che vengono creati all'interno dell'ultimo blocco indiretto doppio parziale.
- **Passo 5 (Somma finale):** Somma tutti i blocchi strutturali utilizzati e aggiungi i blocchi dati. Il totale è dato da:
 $1 (\text{Inode}) + \text{Indiretti singoli base usati} + \text{Indiretti doppi base usati} + (N \cdot \text{Numero indiretti doppi pieni}) + \text{Indiretti singoli usati nell'ultimo doppio} + \text{Blocchi dati}$.

Gerarchia di Memoria (Cache)

Calcolo dell'AMAT (Average Memory Access Time) e Penalità di Miss per Cache a più livelli

Questo algoritmo si applica per trovare i tempi medi di accesso in architetture con cache L1 e L2.

- **Passo 1 (Estrazione parametri):** Identifica i valori forniti dal testo per ogni livello: tempo di accesso (Hit Time_{L1} , Hit Time_{L2}), percentuali di fallimento (Miss Rate_{L1} , Miss Rate_{L2}) e il tempo di accesso alla memoria principale (T_{mem}). Se i tassi sono in percentuale (es. 10%), convertili in formato decimale (0,10) per i calcoli.
- **Passo 2 (Calcolo Penalità di Miss di L1):** La penalità di miss del primo livello (L1) corrisponde al tempo medio necessario per recuperare il dato dai livelli sottostanti (L2 e memoria). Si calcola con la formula:
 $\text{Miss Penalty}_{L1} = \text{Hit Time}_{L2} + (\text{Miss Rate}_{L2} \cdot T_{mem})$.
- **Passo 3 (Calcolo AMAT Totale):** Usa la penalità appena trovata per calcolare il tempo di accesso medio complessivo dell'intero sistema. La formula è:
 $\text{AMAT} = \text{Hit Time}_{L1} + (\text{Miss Rate}_{L1} \cdot \text{Miss Penalty}_{L1})$.

Valutazione del Miglioramento Architeturale

Questo algoritmo si usa quando l'esercizio ti chiede quale modifica (es. ridurre un miss rate o abbassare un tempo di accesso) riduca maggiormente l'AMAT totale.

- **Passo 1 (Scenario Base):** Applica l'algoritmo precedente per calcolare l'AMAT totale con i dati di partenza forniti dall'esercizio.
- **Passo 2 (Simulazione Opzioni):** Per ogni singola opzione di miglioramento proposta (es. "Ridurre L1 miss rate dal 10% al 5%"), sostituisci il nuovo valore nella formula dell'AMAT ricalcolando tutto da capo. Assicurati di testare una sola modifica alla volta, mantenendo inalterati gli altri parametri originali.
- **Passo 3 (Confronto):** Confronta i risultati ottenuti: l'opzione corretta è quella che restituisce il valore numerico di AMAT più basso.

Calcolo dell'Hit Rate minimo per un vincolo di tempo (Reverse Engineering dell'AMAT)

Questo procedimento si utilizza per trovare la percentuale minima di successi in cache necessaria per rispettare una specifica tolleranza di ritardo.

- **Passo 1 (Definizione del Vincolo):** Calcola il tempo massimo accettabile in base alle richieste del testo (es. se il tempo non deve superare del 30% il tempo della cache, il vincolo è $Hit\ Time \cdot 1,30$).
- **Passo 2 (Impostazione Equazione):** Imposta l'equazione dell'AMAT ponendo il Miss Rate come incognita (x). La formula è:
$$Hit\ Time_{cache} + (x \cdot T_{mem}) = Vincolo\ massimo.$$
- **Passo 3 (Risoluzione Incognita):** Risolvi l'equazione per trovare il Miss Rate:
$$x = \frac{Vincolo\ massimo - Hit\ Time_{cache}}{T_{mem}}.$$
- **Passo 4 (Calcolo Hit Rate):** Sottrai il Miss Rate appena trovato dal 100% (o da 1 se usi i decimali) per ottenere la percentuale minima di Hit Rate richiesta. Formula: $Hit\ Rate = 1 - x$.

Scheduling della CPU e Allocazione (Buddy System)

Scheduling della CPU - Shortest Job First (SJF senza prelazione)

- **Passo 1 (Stato iniziale):** Posizionati al tempo $T = 0$ e identifica i processi disponibili verificando il loro tempo di arrivo (Arrival Time).
- **Passo 2 (Scelta del processo):** Tra tutti i processi attualmente pronti, seleziona quello con il tempo di esecuzione stimato (burst time) più breve e mandalo in esecuzione.
- **Passo 3 (Esecuzione ininterrotta):** Poiché l'algoritmo non ha prelazione, fai avanzare il tempo T fino al completamento totale del processo scelto. Traccia quali nuovi processi arrivano nel sistema durante questo lasso di tempo per renderli disponibili al turno successivo.
- **Passo 4 (Calcolo completamento e attesa):** Il *Tempo di Completamento* del processo corrisponde all'unità di tempo successiva alla sua ultima occorrenza, meno il suo tempo di arrivo. Il *Tempo di Attesa* coincide con la prima unità di tempo in cui compare il processo meno il suo tempo di arrivo.
- **Passo 5 (Iterazione):** Ripeti la selezione sul processo più breve tra quelli in coda fino ad esaurimento.

Scheduling della CPU - Round Robin (RR)

- **Passo 1 (Impostazione del quanto):** Identifica il quanto di tempo q specificato dall'esercizio (es. $q = 2$).
- **Passo 2 (Esecuzione parziale):** Preleva il primo processo disponibile in base all'ordine di arrivo ed esegilo per un massimo di q unità di tempo. Se il processo termina prima di q , l'esecuzione si interrompe anticipatamente.
- **Passo 3 (Aggiornamento e accodamento):** Avanza il tempo T . Inserisci nella coda dei processi pronti eventuali nuovi processi arrivati *mentre* il processo corrente era in esecuzione. Successivamente, se il processo interrotto non ha ancora terminato la sua esecuzione, reinseriscilo in fondo alla coda.
- **Passo 4 (Calcolo completamento e attesa):** Al termine di un processo, il *Tempo di Completamento* è l'unità di tempo successiva all'ultima occorrenza meno il tempo di arrivo. Il *Tempo di Attesa* si ottiene contando tutte le unità di tempo antecedenti al completamento (partendo dal tempo di arrivo) in cui il processo in esame non è in esecuzione.

Allocazione della Memoria - Buddy System

- **Passo 1 (Estrazione delle posizioni):** Identifica il numero totale di posizioni disponibili nella memoria indicato dall'esercizio (es. 32 posizioni o 128 posizioni). La grandezza in KB della singola pagina associata a queste posizioni è un dato irrilevante che va ignorato per il conteggio.
- **Passo 2 (Generazione delle liste):** Scrivi la sequenza delle potenze del 2, partendo dal blocco di dimensione 1 e fermandoti al numero totale di posizioni. Ad esempio, per 32 posizioni le dimensioni delle liste sono: 1, 2, 4, 8, 16, 32. Per 128 posizioni: 1, 2, 4, 8, 16, 32, 64, 128.
- **Passo 3 (Conteggio finale):** Il numero esatto di liste di blocchi liberi necessarie al sistema corrisponde alla quantità di numeri appena scritti (es. 6 liste per 32 posizioni, 8 liste per 128 posizioni).
- **Passo 4 (Gestione delle richieste):** Se l'esercizio simula un'allocazione, ricorda che quando viene richiesto un blocco di un certo ordine (r_0) ma è disponibile solo un blocco di dimensione superiore ($f_0 > r_0$), il sistema divide i blocchi ripetutamente a metà fino a creare la dimensione adatta. Alla fine della procedura, viene allocato esattamente un blocco dell'ordine richiesto (r_0).